

Karpathy Makemore Master Notes

A rebuild-from-scratch study note for the project chats

Use this document as a revision map, not as a passive summary. The goal is not to memorize lines from a notebook. The goal is to be able to reconstruct the ideas, code, and reasoning from memory, the way Karpathy builds them in public.

- Read each section once.
- Close the note.
- Recreate the idea on paper or in Python.
- Only then check the note again.

This note is organized around the exact concepts we discussed in the project: tensor shapes, classes and self, the data pipeline, stoi/itos, counts and probabilities, sampling, logits, cross entropy, and the gradient story behind the tiny language model.

1. The whole picture

What this project is really doing: we are building a character-level language model from the ground up. The model sees a short context of characters and predicts the next character. Everything else is just supporting machinery.

- Input: a fixed-size window of previous characters.
- Output: a score for every possible next character.
- Training: increase the score of the correct next character and decrease the others.
- Generation: repeatedly sample the next character from the predicted distribution.

The point of the makemore journey is not only to get a model that works. The point is to understand every layer of the stack: data, tensors, matrices, probability, sampling, loss, gradients, and code structure.

2. How to think like Karpathy

- Start with the simplest possible version.
- Use tiny examples and inspect them.
- Prefer a table, matrix, or picture over abstract words.
- Ask: what is the shape, what is the meaning, and what is the next operation?
- Do not jump to fancy libraries before you understand the raw form.

That is why the project keeps returning to basic Python, simple matrices, and manual reasoning. The aim is not decoration. The aim is transparency.

3. Tensor shapes: the first thing you must see

A tensor is just an n-dimensional array. The shape tells you how many axes there are and how many values live on each axis.

- A scalar has shape ().
- A vector has shape (n,).
- A matrix has shape (rows, columns).
- A 3D tensor has shape (a, b, c).

For a shape like 4 x 8 x 10, do not think of it as one mysterious object. Think of it as a stack of 4 matrices, each matrix having 8 rows and 10 columns. Or depending on the meaning, it could be 4 examples, 8 channels, and 10 features. The numbers only become useful when you attach meaning to each axis.

Rule to remember: the same shape can represent different things in different contexts. Shape is structure; meaning comes from the code and the problem.

4. Why the matrix version matters more than a dictionary

In early counting tasks, a dictionary can store counts. But once you move toward learning and batching, a matrix becomes far more powerful because it is compact, vectorized, and easy to operate on with PyTorch.

- Dictionary: good for readability, but slow and not naturally batch-friendly.
- Matrix: ideal for counting all pairs or transitions at once.
- Matrix operations map directly to linear algebra and neural network code.

That is why the notebook shifts from a dictionary-based mental model to a matrix-based one. The matrix is not just faster. It is the bridge to neural networks.

5. stoi and itos

stoi means string-to-integer. itos means integer-to-string.

- stoi turns a character into its index.
- itos turns an index back into the character.
- This mapping is the bridge between language and tensors.

Example mental model: if '.' maps to 0, 'a' maps to 1, and 'b' maps to 2, then the model can work with numbers while still representing characters. The model never learns letters directly. It learns numbers that stand for letters.

6. The counting matrix

A counting matrix stores how often one character follows another. In the simplest version, row i and column j hold the number of times character i was followed by character j .

This is the first place where the project moves from idea to mechanism.

- Rows represent the current character.
- Columns represent the next character.
- Each cell stores a transition count.

Once you see the matrix this way, the later probability matrix becomes obvious. Count first. Normalize later.

7. torch.zeros, dtype, and indexing

`torch.zeros` creates a tensor filled with zeros. It is the cleanest way to build a counting matrix from scratch.

```
import torch
```

```
N = torch.zeros((27, 27), dtype=torch.int32)
N[0, 1] += 1
```

Why int32 here? Because counts are integers. Do not store counts as floats unless you need to. The exact type should match the meaning.

Indexing with `N[i, j]` means: go to row `i` and column `j`, and update that cell. This is the direct matrix version of saying 'one character followed another character.'

8. Heatmap and visualization

The heatmap is not decoration. It is a debugging tool for your intuition. It lets you see which transitions are common and which are rare.

- `imshow` turns a matrix into a color grid.
- `cmap` controls the color scheme.
- `figsize` controls the size of the picture.
- `plt.text` writes the number inside each cell.

When you write the count inside the cell, you stop treating the matrix as an abstract object and start seeing the actual distribution of the data. That is very important for learning.

9. From counts to probabilities

Counts are not yet probabilities. Probabilities must sum to 1 across the possible next characters.

To convert counts into probabilities, divide each row by the row sum. This gives a normalized distribution for that context.

```
P = N.float()
P = P / P.sum(1, keepdim=True)
```

The `.float()` conversion matters because integer division would not give you proper probabilities. After converting to float, the row sum becomes the denominator of that row.

10. What `keepdim=True` is really doing

This was one of the most important shape ideas in the whole project.

- `P.sum(1)` collapses each row into one number.
- `keepdim=True` keeps the row dimension as size 1.
- That makes broadcasting line up correctly when dividing.

Without `keepdim=True`, the result becomes shape `(27,)`, which can break the intended broadcasting logic or make the code harder to reason about. With `keepdim=True`, the shape stays `(27, 1)`, so each row divides by its own row sum cleanly.

In plain words: keepdim=True preserves the structure of the matrix while still computing the sum.

11. Broadcasting

Broadcasting is the automatic expansion of a smaller tensor to match a larger tensor during arithmetic. It saves you from writing manual loops.

- A row vector can be applied across many rows.
- A column vector can be applied across many columns.
- Broadcasting must match the intended axes.

The core habit is to always ask: which axis is being reduced, and which axis should remain visible for broadcasting?

12. Sampling with torch.multinomial

The probability row tells you how likely each next character is. But generation needs an actual choice, not just probabilities. That is where torch.multinomial comes in.

- It samples an index according to probabilities.
- Higher probability means higher chance of being chosen.
- It prevents generation from getting stuck in a repetitive greedy loop.

This is important: choosing the maximum every time is deterministic and often boring. Sampling introduces controlled randomness, which makes generation more natural.

Random seed matters because it makes results repeatable while you debug. Same seed, same sampled sequence.

13. The generation loop

The generation loop is simple once the pieces are in place.

```
ix = 0 # start token
out = []

while True:
    p = P[ix]
    ix = torch.multinomial(p, num_samples=1, replacement=True).item()
    out.append(itos[ix])
    if ix == 0:
        break
```

Interpretation: start from the beginning token, get the probability distribution for the next character, sample one character, convert the index back to a character, append it, and stop when the end token appears.

That loop is the whole model in action. Training makes the probabilities good. Sampling turns probabilities into text.

14. build_dataset: the sliding window idea

The build_dataset function is where raw text becomes training examples.

- Choose a block size, such as 3.
- Take the previous 3 characters as context.
- Predict the next character after that context.
- Slide one character at a time through the word.

The block size is the amount of history the model can see. If block_size = 3, then the model always looks at the last 3 characters to predict the next one.

This is why the dataset is built by sliding a window across each word and collecting many (context, target) pairs.

```
context = [0] * block_size
X, Y = [], []

for word in words:
    for ch in word + '.':
        ix = stoi[ch]
        X.append(context)
        Y.append(ix)
        context = context[1:] + [ix]
```

The important idea is not the exact syntax. The important idea is that each training example is a local prediction problem.

15. Why block size matters

- Small block size: model sees little context.
- Larger block size: model sees more history.
- Too large can make learning heavier and data sparser.

Block size is a design tradeoff. More context can help, but only if the model and data can support it. The project uses a small fixed block to keep the reasoning clear.

16. Python classes: what self really means

In Python, self is the current object. It is how methods access the object's own data.

- self.weight means the weight stored inside this object.
- self.bias means the bias stored inside this object.
- The object carries its parameters with it.

The constructor `__init__` runs when the object is created. It sets up the internal state. The `__call__` method lets the object act like a function.

```
class Linear:
    def __init__(self, fan_in, fan_out, bias=True):
        self.weight = torch.randn((fan_in, fan_out)) / fan_in**0.5
        self.bias = torch.zeros(fan_out) if bias else None

    def __call__(self, x):
        out = x @ self.weight
        if self.bias is not None:
            out = out + self.bias
        return out
```

The object is a container for parameters plus behavior. This is exactly how neural network layers are built.

17. The `parameters()` method

When a layer stores trainable tensors, you need a simple way to hand them to the optimizer. `parameters()` returns the list of tensors that should be updated during training.

```
def parameters(self):
    return [self.weight]
```

The second line means: return a Python list containing the weight tensor. If bias exists and is trainable, it should also be included in the list.

Why a list? Because the optimizer expects a collection of tensors, not a single tensor hidden inside the object.

18. Linear layer intuition

A linear layer does two things: multiplies by a weight matrix and optionally adds bias.

- Input x has features.
- Weight matrix transforms those features.
- Bias shifts the result.

This is the basic building block of the neural network. Everything more advanced still relies on this transformation idea.

19. Why initialization matters

Weights are usually initialized with a scaling rule such as division by $\sqrt{\text{fan_in}}$. The reason is to keep activations from exploding or collapsing too quickly.

- Too large: activations blow up.
- Too small: activations vanish.
- Careful initialization keeps training stable.

You do not need to memorize the formula before the intuition. The intuition is: preserve signal magnitude as data flows through layers.

20. Logits: raw scores before probability

Logits are the raw outputs of the model before softmax. They are not probabilities yet.

- Higher logit means the model is leaning more toward that class.
- Logits can be any real number.
- Softmax turns logits into probabilities.

A common confusion is thinking the model is already predicting probabilities. Not yet. It is predicting scores. Softmax converts those scores into a normalized distribution.

21. Softmax intuition

Softmax takes a vector of scores and converts it into a probability distribution.

- Large scores become larger probabilities.
- Small scores become smaller probabilities.
- All probabilities sum to 1.

Softmax is the mathematical bridge from raw model confidence to usable probabilities. But for numerical stability, the implementation often works on logits directly inside the loss function.

22. Cross entropy intuition

Cross entropy measures how well the predicted distribution matches the true target. If the model assigns high probability to the correct class, the loss is small. If it assigns low probability, the loss is large.

- Good prediction -> low loss.
- Bad prediction -> high loss.
- The loss pushes the model to sharpen the correct answer.

This is not just a formula to memorize. It is a punishment mechanism for wrong confidence.

When the model is very wrong, the loss becomes large. When the model is confidently right, the loss becomes small.

23. Why logits and cross entropy are used together

Using logits directly in the loss is more stable and more convenient than manually applying softmax first and then computing log probabilities.

- The implementation is numerically safer.

- The gradients behave better.
- The code is cleaner.

In practice, this is why PyTorch provides a combined cross entropy function. It handles the stable version internally.

24. The gradient story: what is actually happening

Training means changing the weights so the correct next character becomes more likely.

- Forward pass: compute logits and loss.
- Backward pass: compute gradients.
- Update step: adjust weights in the opposite direction of the gradients.

The gradient tells you how much each parameter contributed to the loss. If a parameter makes the loss worse, gradient descent pushes it in the opposite direction.

25. Why dlogits, dnormlogits, clone, and ones_like matter

These kinds of variables appear when you manually derive backprop. The names look strange at first, but they are just tracking derivatives through each operation.

- dlogits means gradient with respect to logits.
- dnormlogits means gradient after normalization or a transformed version of logits.
- clone() creates a separate tensor so you can safely modify one version without changing the original.
- ones_like creates a tensor of ones with the same shape, often used as a starting gradient pattern.

The reason one place used clone and another used ones_like was not magic. It was about whether you needed a copy of an existing tensor or a fresh tensor of the same shape filled with ones.

Manual backward passes are often about bookkeeping. The challenge is not the math alone. It is keeping the chain of operations straight.

26. Why the model output changes through each layer

Each layer transforms the tensor. A linear layer changes the representation. A nonlinearity changes the shape of the signal. The final layer produces logits for the output classes.

- Input context becomes embeddings or one-hot vectors.
- Linear transform mixes information.
- Activation adds nonlinearity.
- Final scores represent class preference.

Do not think of the network as one monolithic block. Think of it as a sequence of controlled transformations.

27. Common confusion points we cleared up

- Why a matrix is better than a dictionary for model work.
- Why `keepdim=True` is needed in row normalization.
- Why `.float()` is needed before dividing counts.
- Why sampling is not the same as always taking `argmax`.
- Why logits are not probabilities.
- Why `self` appears in class methods.
- Why `parameters()` returns a list.

These are not separate topics. They are all part of one mental pipeline from text to tensors to learning to generation.

28. The rebuild-from-scratch checklist

1. Write `stoi` and `itos` from memory.
2. Create the counting matrix with `torch.zeros`.
3. Fill counts using indexes from training text.
4. Visualize the matrix as a heatmap.
5. Normalize rows into probabilities.
6. Sample from a row using `torch.multinomial`.
7. Explain what `keepdim=True` does.
8. Build the dataset with a sliding context window.
9. Write a `Linear` class with `__init__`, `__call__`, and `parameters()`.
10. Explain logits, softmax, and cross entropy in plain language.
11. Trace one backward pass by hand.

If you can do these steps without checking the note, you are not just memorizing Karpathy. You are learning the method.

29. Practice prompts

- Explain a $4 \times 8 \times 10$ tensor in two different contexts.
- Describe why a matrix transition table is more useful than a dictionary.
- Show what happens if `keepdim=True` is removed.
- Write the generation loop without looking.
- Explain the difference between logits and probabilities to a beginner.
- Describe `parameters()` in one sentence and one code example.

Use these prompts as self-tests. If you can answer them cleanly, the understanding is becoming real.

30. Final mental model

Karpathy-style learning is about stripping away mystery. Start with counts. Turn counts into probabilities. Turn probabilities into sampled text. Replace hand-built counts with learnable weights. Replace hand-written rules with gradient descent. Keep every tensor shape visible. Keep every operation understandable.

That is the path from raw text to a working character model. The note is meant to help you reconstruct that path from memory, not just recognize it on a page.

Appendix A: Key snippets to remember

```
# probability normalization
P = N.float()
P = P / P.sum(1, keepdim=True)

# sampling
ix = torch.multinomial(P[ix], num_samples=1, replacement=True).item()

# linear layer
out = x @ weight + bias

# parameter collection
def parameters(self):
    return [self.weight]
```

Do not memorize these as isolated lines. Memorize the reason each line exists.

Appendix B: One-sentence meanings

- `stoi`: convert character to index.
- `itos`: convert index to character.
- `logits`: raw class scores.
- `softmax`: convert scores into probabilities.
- `cross entropy`: how wrong the predicted distribution is.
- `broadcasting`: automatic shape matching.
- `keepdim=True`: preserve the reduced axis as size 1.